

1. Explain HTML Basic Tags with Examples

HTML (HyperText Markup Language) is a standard markup language used to create and structure web pages. An HTML document consists of different **tags**, which instruct the browser how to display the content.

Basic structure of an HTML document

```
<!DOCTYPE html>
<html>
<head>
  <title>My Web Page</title>
</head>
<body>
  <h1>Welcome to HTML</h1>
  <p>This is a sample web page.</p>
</body>
</html>
```

Important HTML tags

1. **<html>** tag:

It is the root element of an HTML document. All other HTML elements are placed inside it.

2. **<head>** tag:

It contains information about the document, such as the title, metadata, CSS links, and scripts.

3. **<title>** tag:

It specifies the title displayed in the browser tab.

```
<title>Web Programming</title>
```

4. **<body>** tag:

It contains the actual content visible to the user.

5. **Heading tags:**

HTML provides six heading levels from **<h1>** to **<h6>** .

```
<h1>Main Heading</h1>
<h2>Sub Heading</h2>
```

6. Paragraph tag:

The `<p>` tag defines a paragraph.

```
<p>This is a paragraph.</p>
```

7. Line break and horizontal rule:

```
<br>
<hr>
```

`<br>` inserts a line break, while `<hr>` creates a horizontal line.

8. Hyperlink tag:

```
<a href="page.html">Visit Page</a>
```

The `<a>` tag creates a hyperlink.

9. Image tag:

```

```

The `<img>` tag is used to display an image.

Thus, HTML tags provide the basic structure required to create a web page.

2. Write Short Notes on Lists, Forms, Tables and Frames in HTML

Lists

HTML lists are used to organize related information in a structured manner.

Ordered List

An ordered list displays items in a particular sequence.

```
<ol>
  <li>HTML</li>
```

```
<li>CSS</li>
<li>JavaScript</li>
</ol>
```

Unordered List

An unordered list displays items with bullets.

```
<ul>
  <li>HTML</li>
  <li>CSS</li>
</ul>
```

Description List

A description list is used to define terms and their descriptions.

```
<dl>
  <dt>HTML</dt>
  <dd>Language used for web pages.</dd>
</dl>
```

Forms

An **HTML form** is used to collect information from the user and submit it for processing.

```
<form action="process.php" method="post">
  Name:
  <input type="text" name="username">

  <input type="submit" value="Submit">
</form>
```

The important attributes of the `<form>` element include:

- **ACTION** — specifies the processing destination.
- **METHOD** — specifies the data submission method, such as GET or POST.
- **ENCTYPE** — specifies the encoding of submitted data.

Common form controls include text fields, password fields, radio buttons, checkboxes, buttons, text areas, and selection lists.

Tables

HTML tables are used to arrange data in **rows and columns**.

Important tags include:

- `<table>` — defines the table.
- `<tr>` — defines a table row.
- `<th>` — defines a table heading.
- `<td>` — defines a table data cell.

```
<table border="1">
  <tr>
    <th>Name</th>
    <th>Marks</th>
  </tr>
  <tr>
    <td>Sandeep</td>
    <td>85</td>
  </tr>
</table>
```

Tables are commonly used to display structured information such as marks, schedules, and reports.

Frames

Frames were used in older HTML to divide a browser window into multiple sections, with each section displaying a separate HTML document.

```
<frameset cols="30%,70%">
  <frame src="menu.html">
  <frame src="content.html">
</frameset>
```

Each frame can display independent content. However, frames are now considered obsolete and are not supported in HTML5.

3. Discuss CSS Types and its Applications

CSS (Cascading Style Sheets) is used to control the appearance and presentation of HTML documents. CSS allows developers to define colors, fonts, borders, spacing, and page layout.

Types of CSS

1. Inline CSS

Inline CSS is applied directly to an HTML element using the `style` attribute.

```
<p style="color: blue; font-size: 20px;">  
  Web Programming  
</p>
```

It is useful when a style needs to be applied to one specific element.

2. Internal CSS

Internal or embedded CSS is written inside the `<style>` element in the `<head>` section.

```
<head>  
<style>  
  h1 {  
    color: blue;  
    font-family: Arial;  
  }  
</style>  
</head>
```

It is suitable when the same style is required for multiple elements within one web page.

3. External CSS

External CSS is stored in a separate file with the `.css` extension.

```
<link rel="stylesheet" href="style.css">
```

Example CSS file:

```
h1 {  
  color: blue;
```

```
}
```

External CSS allows the same stylesheet to be applied to multiple web pages and is therefore useful for large websites.

Applications of CSS

CSS is used for:

- Changing text and background colors
- Setting fonts and font sizes
- Adding borders
- Controlling margins and padding
- Designing web-page layouts
- Formatting HTML elements
- Creating consistent styles across a website

Thus, CSS separates the **presentation of a web page** from its **HTML structure and content**, improving maintainability and design consistency.

4. Describe the HTML5 Features and Web Creation Tools

HTML5 is an advanced version of HTML developed to support modern web applications, multimedia, graphics, and interactive content.

Major Features of HTML5

1. Simplified Document Structure

HTML5 provides a simple document declaration:

```
<!DOCTYPE html>
```

It also supports modern character encoding and page structure.

2. Semantic Elements

HTML5 provides meaningful structural elements such as:

- <header>
- <footer>
- <article>
- <section>
- <nav>

These elements improve the structure and readability of web documents.

3. Canvas

The `<canvas>` element provides a drawing area that can be controlled using JavaScript.

It can be used to create:

- Shapes
- Lines and arcs
- Text
- Graphics
- Animation
- Web games

4. Multimedia Support

HTML5 supports multimedia using elements such as:

```
<video controls>
  <source src="video.mp4">
</video>
```

It provides native support for audio and video without relying on traditional browser plug-ins.

5. Events and Interaction

HTML5 works with JavaScript events to create interactive web applications. Examples include mouse events, change events, and timed events.

Web Creation Tools

- **Box2D**
- **Cocos2D**
- **PhoneGap**
- **AppMobi**
- **Impact.js**

These tools are associated with developing web, mobile, graphical, and game applications.

Thus, HTML5 provides a powerful platform for creating modern, interactive, and multimedia-based web applications.

5. Explain JavaScript Events with Examples

A **JavaScript event** is an action or occurrence detected by the browser that can trigger the execution of a JavaScript program.

An event may be generated by:

- User interaction
- Mouse actions
- Keyboard actions
- Changes in form elements
- Browser or document activity

Common Events

1. Click Event

The `onclick` event occurs when an element is clicked.

```
<button onclick="showMessage()">
  Click Me
</button>

<script>
function showMessage() {
  alert("Button Clicked");
}
</script>
```

2. Change Event

The `onchange` event occurs when the value of an input control changes.

```
<input type="range"
  onChange="changescale(this.value)">
```

3. Mouse Events

Common mouse events include:

- `onclick`
- `onmousedown`
- `onmouseup`
- `onmousemove`
- `onmouseover`

These events are useful for creating interactive applications. The faculty material uses mouse events for activities such as dragging graphical objects.

4. Event Listener

Events can also be handled using `addEventListener()` .

```
button.addEventListener("click", showMessage);
```

Example from the faculty material:

```
v.addEventListener("ended", restart, false);
```

Here, the `restart` function is executed when the video ends.

Thus, JavaScript events make web pages dynamic and interactive by allowing the program to respond to user actions and browser activity.

6. Explain BOM and DOM in JavaScript

BOM — Browser Object Model

The **Browser Object Model (BOM)** allows JavaScript to interact with the web browser and its environment.

The primary BOM object is:

```
window
```

Important BOM objects include:

- `window`
- `navigator`
- `location`
- `history`
- `screen`

Example

```
window.alert("Welcome");
```

Another example is:

```
history.back();
```

This method takes the user to the previous page in browser history.

DOM — Document Object Model

The **Document Object Model (DOM)** represents an HTML document as a collection or tree of objects. It allows JavaScript to access, modify, create, and manipulate web-page elements.

Example

```
<p id="demo">Old Text</p>

<script>
document.getElementById("demo").innerHTML =
    "New Text";
</script>
```

JavaScript accesses the element using its ID and modifies its content.

Difference between BOM and DOM

BOM	DOM
Represents the browser environment.	Represents the HTML document.
Main object is <code>window</code> .	Main object is <code>document</code> .
Controls browser-related functions.	Controls document elements and content.

Thus, **BOM interacts with the browser**, while **DOM interacts with the web document**.

7. What is HTTP? Explain HTTP Request and Response Formats

HTTP (HyperText Transfer Protocol) is an application-layer protocol used for communication between a **web client** and a **web server**.

The communication follows a **request-response model**:

```
Client  HTTP Request  Server
```

Client HTTP Response Server

HTTP Request

An HTTP request is sent by the client to request a resource or submit data to the server.

General format

Request-Line
Header Fields

Message Body

Example

```
GET /index.html HTTP/1.1
Host: www.example.com
Accept: text/html
```

Request Components

1. Request Line

```
GET /index.html HTTP/1.1
```

It contains:

- HTTP method
- Resource path
- HTTP version

2. Headers

Headers provide additional information about the request.

3. Message Body

The body may contain data sent to the server, especially with POST requests.

Common HTTP methods include:

- GET — retrieves a resource.
- POST — submits data.
- PUT — sends or updates data.

- **DELETE** — requests deletion.
-

HTTP Response

An HTTP response is generated by the server and returned to the client.

General format

```
Status-Line  
Header Fields  
  
Message Body
```

Example

```
HTTP/1.1 200 OK  
Content-Type: text/html  
  
<html>  
  <body>Welcome</body>  
</html>
```

Response Components

- 1. Status Line:** Contains HTTP version, status code, and status message.
- 2. Headers:** Provide information about the returned content.
- 3. Response Body:** Contains the requested resource.

Common status codes include:

- **200 OK** — Request successful.
- **404 Not Found** — Resource not found.
- **500 Internal Server Error** — Server error.

Thus, HTTP enables standardized communication between browsers and web servers.

8. Explain JavaScript Functions

A **JavaScript function** is a named and reusable block of code designed to perform a specific task.

Functions improve program organization by dividing a large program into smaller and manageable units.

Function Syntax

```
function functionName(parameters) {  
    // statements  
}
```

Example

```
function displayMessage() {  
    document.write("Welcome to JavaScript");  
}  
  
displayMessage();
```

The function is first **defined** and then **called**.

Functions with Parameters

A function may receive values called **parameters**.

```
function add(a, b) {  
    var sum = a + b;  
    document.write(sum);  
}  
  
add(10, 20);
```

Here:

- `a` and `b` are parameters.
- `10` and `20` are arguments.

Function with Return Value

A function can return a value to the calling program using the `return` statement.

```
function multiply(a, b) {  
    return a * b;  
}
```

```
var result = multiply(5, 4);
```

Advantages of Functions

Functions provide:

- **Reusability** — the same code can be called repeatedly.
 - **Modularity** — a program can be divided into smaller units.
 - **Maintainability** — code is easier to modify and debug.
 - **Reduced code duplication** — repeated statements can be avoided.
 - **Better program organization.**
-